

MANUAL DE CONFIGURACIÓN Y OPERACIÓN CON VARIABLES DE ENTORNO

DESPLIEGUE CENTRALIZADO CON ARCHIVO .ENV E INSTRUCCIONES DEL CICLO DE VIDA EN BASH

1. Arquitectura Centralizada con Archivo .env

El uso de un archivo .env en la raíz del proyecto permite desacoplar los secretos de infraestructura y las credenciales de la base de datos de los archivos de configuración de Docker Compose. Esto centraliza el mantenimiento y previene la exposición accidental de claves de acceso en el sistema de control de versiones Git.

Al invocar comandos, Docker Compose lee automáticamente el archivo .env presente en el directorio actual e inyecta dichas variables tanto en el entorno del orquestador como dentro de las variables de entorno de los contenedores que lo requieran.

2. Contenido de los Archivos de Configuración Modificados

A. Archivo de Variables de Entorno (.env)

Crea este archivo en la raíz del proyecto (C:\AppDesarrollo\CursoProyectos\nginx_facultad\.env):

```
# Configuración de Base de Datos para MariaDB (Interno)
MARIADB_DATABASE=facultad
MARIADB_USER=facultad_user
MARIADB_PASSWORD=facultad_pass
MARIADB_ROOT_PASSWORD=root_pass

# Puertos Expuestos hacia el Host (Windows)
PUERTO_EXTERNO_NGINX=9090
PUERTO_EXTERNO_MARIADB=3307

# Parámetros de Conexión del Backend (FastAPI habla con MariaDB de forma interna)
DB_HOST=db
DB_PORT=3306
DB_NAME=facultad
DB_USER=facultad_user
DB_PASS=facultad_pass
```

B. Archivo de Orquestación Dinámico (docker-compose.yml)

Modificado para consumir de forma nativa las directivas declaradas en el archivo .env:

```
services:
  db:
    image: mariadb:10.11
    container_name: mariadb-facultad
    restart: always
    environment:
```

```

    MARIADB_DATABASE: ${MARIADB_DATABASE}
    MARIADB_USER: ${MARIADB_USER}
    MARIADB_PASSWORD: ${MARIADB_PASSWORD}
    MARIADB_ROOT_PASSWORD: ${MARIADB_ROOT_PASSWORD}
  ports:
    - "${PUERTO_EXTERNO_MARIADB}:3306"
  volumes:
    - mariadb_data:/var/lib/mysql
  healthcheck:
    test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
    interval: 5s
    timeout: 5s
    retries: 10
    start_period: 5s

app:
  build: .
  container_name: fastapi-facultad
  restart: always
  volumes:
    - ./app
  environment:
    - DB_HOST=${DB_HOST}
    - DB_PORT=${DB_PORT}
    - DB_NAME=${DB_NAME}
    - DB_USER=${DB_USER}
    - DB_PASS=${DB_PASS}
  depends_on:
    db:
      condition: service_healthy

nginx:
  image: nginx:latest
  container_name: nginx-facultad
  restart: always
  ports:
    - "${PUERTO_EXTERNO_NGINX}:80"
  volumes:
    - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
  depends_on:
    - app

volumes:
  mariadb_data:

```

3. Ciclo de Vida del Entorno: Comandos de Bash / PowerShell

A continuación se describen detalladamente los procedimientos de operación para la administración completa de los microservicios, utilizando la interfaz de comandos estándar.

A. Montar y Construir el Entorno por Primera Vez

¿Para qué? Descarga las imágenes base necesarias, compila el entorno limpio de Python aislado leyendo el Dockerfile, genera el volumen físico persistente en disco y asocia la red virtual dockerizada.

Comando:

```
docker compose up -d --build
```

Nota: El modificador --build asegura que si se agregaron nuevos paquetes al archivo requirements.txt, la imagen de FastAPI se recompilará desde cero aplicando los cambios.

B. Levantar / Iniciar el Entorno Existente

¿Para qué? Encender los servicios que previamente fueron creados o detenidos, respetando de manera estricta la cola del Healthcheck (FastAPI esperará a que MariaDB esté en estado saludable antes de lanzar Uvicorn).

Comando:

```
docker compose up -d
```

Nota: El flag -d (detached) ejecuta los contenedores en segundo plano, devolviendo inmediatamente el control de la terminal al usuario.

C. Detener los Contenedores Temporalmente

¿Para qué? Pausar la ejecución de los servicios para liberar memoria RAM y recursos de CPU de la máquina host sin alterar ni destruir el estado de los contenedores ni la información interna.

Comando:

```
docker compose stop
```

D. Reiniciar los Contenedores

¿Para qué? Aplicar de manera inmediata cambios realizados en el código fuente de Python, modificaciones en las vistas HTML de la carpeta templates/ o alteraciones menores de configuración.

Comando:

```
docker compose restart
```

E. Bajar y Limpiar el Entorno

¿Para qué? Detener los procesos y **eliminar** por completo los contenedores, las redes internas mapeadas y los accesos creados. Ideal para realizar un mantenimiento limpio del sistema.

Comando:

```
docker compose down
```

⚠ *Importante: Este comando NO borra los datos de los alumnos, puesto que el volumen mariadb_data se declara externo e independiente a los ciclos de vida de destrucción del contenedor.*

F. Consultar Estados y Diagnósticos

Para comprobar la sincronización del Healthcheck y el estado de los servicios:

```
docker compose ps
```

Para visualizar en tiempo real la salida de logs y trazas de error de la aplicación Python:

```
docker compose logs -f app
```



Recordatorio de Seguridad para Git

Dado que el archivo `.env` contiene contraseñas de producción y credenciales del sistema gestor de bases de datos, se debe incluir explícitamente dentro de tu archivo `.gitignore` para evitar su publicación en repositorios públicos.